

Apontamento JERP — Inspeção e Embalagem

Sistema: gde-insp-embalagem

Data: 07/07/2026

Versão do código: implementação atual em src/

Resumo: O apontamento de produção no JERP ocorre *uma vez por caixa embalada*, após a inspeção visual de todos os blisters. A modificação recente inclui o array `embalagens` no payload, com um objeto `{ barcode }` por blister embalado — código QR lido na inspeção. A quantidade oficial vem sempre do banco local, nunca do frontend.

Índice

1. Visão geral do ciclo
2. Integrações com o JERP
3. Preparação da OP
4. Fluxo de inspeção por caixa
5. Apontamento — payload e códigos de barra
6. Variáveis de ambiente e tipos
7. Dados no banco local
8. Validações e auditoria
9. Fluxos especiais
0. Referência de arquivos
1. Exemplo completo

1. Visão geral do ciclo

O sistema controla inspeção visual de caixas, blisters e peças (via worker de visão) e, ao final de cada caixa, realiza o **apontamento de produção no JERP** — único momento em que dados de produção são **escritos** no ERP.

```
Sincronizar OP (JERP → local)
  → Inspecionar caixa / blisters / peças
  → Persistir no PostgreSQL (packedAt + code QR)
  → Apontar no JERP (local → JERP) com embalagens[].barcode
  → Salvar idBarras na caixa
  → Imprimir etiqueta (pdfBase64)
  → Próxima caixa
```

O apontamento **não** acontece a cada blister. Ocorre quando a inspeção da caixa termina e a etiqueta é solicitada.

2. Integrações com o JERP

2.1 Leitura GET

Usadas para sincronizar OP, validar quantidades e reconciliar pendências. Nenhum dado de produção é enviado.

Método	Endpoint	Função	Implementação
GET	{JERP_API}/ordemproducao/{id}	Busca OP pelo ID interno JERP	getOpFromId()
GET	{JERP_API}/ordemproducao/{numero}	Busca OP pelo número visível	getOpFromCode()
GET/POST	{JERP_API_AUTH}/users/emails/...	Autenticação de operador	jerp-auth.ts

getOpFromRef(ref) tenta primeiro por ID interno; se falhar, tenta por número da OP.

2.2 Escrita POST

Única operação que registra produção no JERP:

Método	Endpoint	Função	Implementação
POST	{JERP_API}/ordemproducao	Apontamento de caixa + geração de etiqueta	generateBarcode()

3. Preparação da OP (antes da inspeção)

Ao abrir /op/{opId}, o sistema chama syncAndGetOpToProduceById:

1. Consulta OP no JERP (getOpFromRef)
2. Se OP não existe localmente: valida (validateOpJerpToProduce), seleciona embalagens (selectOpPackagings), cria ProductType, BoxType, BlisterType, Op, OpBox e OpBoxBlister
3. Planeja caixas a partir de quantidadeAProduzir do JERP
4. Se não há caixas pendentes: reconcilia quantidade local com JERP

3.1 Fórmulas de planejamento

```

blisters totais = ceil(quantidadeAProduzir / slots)
caixas totais   = ceil(blisters / limitPerBox)
última caixa    = pode ter menos blisters; último blister pode ter menos peças

```

Exemplo: 1.080 peças, 6 slots, 8 blisters/caixa → 180 blisters → 23 caixas (última com 4 blisters × 6 = 24 peças).

3.2 Reconciliação de quantidade

```

quantityToProduce (local) = peças já embaladas (banco) + quantidadeAProduzir (JERP)

```

4. Fluxo de inspeção por caixa

Tela: `src/app/op/[opId]/page.tsx` — 4 etapas (step):

4.1 Finalização da caixa (gatilho do apontamento)

Quando todos os blisters da caixa foram inspecionados:

1. `persistBoxStatusWithBlisters(boxId, blisters)` grava:

- `OpBoxBlister.packedAt` e `OpBoxBlister.code` (QR real)
- `OpBox.packedAt` e `OpBox.status = PACKAGED`

2. Após 2 segundos: `printTag(boxId)` → `POST /api/op-jerp/barcode`

Até este ponto nada foi enviado ao JERP. Toda inspeção fica no banco local e no worker de visão.

4.2 Sequência do apontamento (fluxo normal)

```
Operador finaliza último blister da caixa
→ persistBoxStatusWithBlisters (PostgreSQL)
→ printTag (frontend)
  → fetchAuthoritativePackedBoxSummary (server action)
  → POST /api/op-jerp/barcode
    → getAuthoritativeBoxPackedSummary (banco)
    → buildJerpEmbalagemApontamento (monta embalagens[])
    → generateBarcode → POST JERP /ordemproducao
    → saveTagId (OpBox.barCode)
    → OpActivityLog (auditoria)
→ Diálogo de impressão (idBarras + pdfBase64)
```

5. Apontamento — payload e códigos de barra

5.1 Modificação principal: array `embalagens`

Antes, o payload enviava apenas `id`, `quantidadeApontada` e `userName`. Agora inclui `embalagens`: um item por blister embalado, cada um com o código QR lido na inspeção.

```
{
  "id": 430505,
  "quantidadeApontada": 48,
  "userName": "operador@gde.com.br",
  "embalagens": [
    { "barcode": "0710380009" },
    { "barcode": "0710380006" },
    { "barcode": "0710380005" }
  ]
}
```

5.2 Montagem do payload

Função central: `buildJerpEmbalagemApontamento()` em `build-jerp-embalagem-apontamento.ts`

```
blisters.map((blister) => ({ barcode: blister.code })))
```

- **Origem do barcode:** `OpBoxBlister.code` — QR lido na etapa 1 da inspeção
- **Ordem:** `packedAt ASC` (ordem de embalagem)
- **Quantidade por blister:** usada apenas para calcular `quantidadeApontada`; *não* vai no payload JERP
- **Filtro:** apenas blisters com `packedAt IS NOT NULL`

5.3 Campos do payload

Step	Etapas	Validação
0	Caixa	Tipo e modelo corretos; exatamente 1 caixa
1	Blister	Tipo, QR do blister, OP correta, não duplicado
2	Quantidade	Contagem de peças (visão) vs. planejado
3	Etiqueta	Geração e impressão via JERP

Campo	Tipo	Origem	Descrição
<code>id</code>	number	<code>Op.id</code> (ID interno JERP)	Identificador da ordem de produção no ERP
<code>quantidadeApontada</code>	number	Soma de <code>OpBoxBlister.quantity</code> com <code>packedAt</code>	Total de peças apontadas nesta caixa
<code>userName</code>	string	E-mail da sessão NextAuth	Operador responsável
<code>embalagens</code>	array	<code>buildJerpEmbalagemApontamento()</code>	Um objeto por blister embalado
<code>embalagens[].barcode</code>	string	<code>OpBoxBlister.code</code>	Código QR do blister (ex.: 07285300651)

5.4 O que NÃO vai no payload

Dado	Motivo
<code>OpBox.id</code> (caixa local)	Uso interno apenas
<code>clientQuantity</code> do frontend	Apenas auditoria; ignorado no apontamento
Quantidade por blister individual	JERP recebe só barcode; total em <code>quantidadeApontada</code>
Tipo caixa/blister/produto	Já conhecidos pelo JERP na OP
Imagens / status inspeção	Permanecem no worker e log local
<code>pdfBase64</code>	É retorno do JERP, não envio

5.5 Resposta do JERP (PrintTagJerpDto)

```
{
  "message": "...",
  "id": 430505,
  "quantidadeApontada": 48,
  "idBarras": 7285300651,
  "quantidadePendente": 1032,
  "descricao": "TL-06-2-0007_02",
  "pdfBase64": "<PDF da etiqueta em Base64>"
}
```

Após sucesso: saveTagId grava OpBox.barCode e OpBox.barCodeGeneratedAt.

5.6 Fonte autoritativa dos dados

A quantidade e lista de blisters **nunca** vêm do cliente para o apontamento:

```
getAuthoritativeBoxPackedSummary(boxId)
→ getPackedBlistersByBox(boxId) // WHERE packedAt IS NOT NULL
→ quantity = soma(blister.quantity)
```

O campo clientQuantity enviado pela UI serve apenas para detectar divergência cliente × banco. Se divergir, usa o banco e registra alerta no log.

5.7 Rotas e gatilhos

Gatilho	Caminho	Auditoria OpActivityLog
Fluxo normal de inspeção	printTag() → POST /api/op-jerp/barcode	Sim (BARCODE_GENERATED)
Reimpressão painel diário	generateBarcodeByBoxId() → generateBarcode() direto	Não

6. Variáveis de ambiente e tipos

6.1 Variáveis de ambiente

Variável	Obrigatória	Uso
JERP_API	Sim	Base URL da API JERP (GET OP, POST apontamento)
JERP_TOKEN	Sim	Bearer token para autenticação nas chamadas JERP
JERP_API_AUTH	Sim (login)	Base URL para autenticação de usuários

Definidas em `.env` / `.env.example`. Validadas em `src/shared/services/jerp/index.ts` na inicialização.

6.2 Tipos TypeScript

Tipo	Arquivo	Campos relevantes
JerpEmbalagemApontamentoDto	jerp-embalagem-apontamento-dto.ts	barcode: string
BlisterApontamentoSource	build-jerp-embalagem-apontamento.ts	code, quantity
PrintTagJerpDto	print-tag-jerp-dto.ts	Resposta JERP (idBarras, pdfBase64, etc.)
AuthoritativeBoxPackedSummary	get-authoritative-box-packed-summary.ts	boxId, quantity, blisterCount, blisters[]
GenerateBarcodeBody	api/op-jerp/barcode/route.ts	opId, boxId, clientQuantity?

JerpBlisterApontamentoDto e buildJerpApontamentoPayload estão deprecados — usar JerpEmbalagemApontamentoDto e buildJerpEmbalagemApontamento.

7. Dados no banco local (PostgreSQL)

Tabela / Campo	Momento	Valor
OpBoxBlister.code	Inspeção etapa 1	QR do blister lido pelo operador/visão
OpBoxBlister.packedAt	Inspeção etapa 2 (aprovação)	Timestamp de embalagem do blister
OpBoxBlister.quantity	Planejamento da OP	Peças neste blister (usado na soma)
OpBox.packedAt	Finalização da caixa	Timestamp de embalagem da caixa
OpBox.status	Finalização	PACKAGED ou PACKAGED_W_BREAK
OpBox.barCode	Após JERP	idBarras retornado pelo JERP
OpBox.barCodeGeneratedAt	Após JERP	Timestamp da geração da etiqueta
OpActivityLog.details	Após apontamento (rota barcode)	Payload embalagens, resposta JERP (sem pdfBase64)

8. Validações e auditoria

Verificação	Onde	Ação se falhar
Sessão autenticada	/api/op-jerp/barcode	HTTP 401
opId e boxId presentes	Rota barcode	HTTP 400
packedSummary.quantity > 0	Rota barcode	HTTP 400 — nenhum blister embalado
JERP retorna erro	generateBarcode	HTTP com status da API JERP
tag.quantidadeApontada !== authoritativeQuantity	Rota barcode	HTTP 502 — divergência banco × JERP
clientQuantity !== authoritativeQuantity	Rota barcode	Log de alerta; usa banco

8.1 Log de auditoria (OpActivityLog)

Registrado na rota /api/op-jerp/barcode com:

- event: "BARCODE_GENERATED"
- embalagens — array enviado ao JERP
- authoritativeQuantity, clientQuantity
- idBarras, quantidadePendente
- jerpResponse — sem pdfBase64

9. Fluxos especiais

9.1 Quebra de caixa (PACKAGED_W_BREAK)

Supervisor autoriza finalizar caixa antes de preencher todos os blisters planejados. Remove blisters não embalados, persiste os embalados, marca PACKAGED_W_BREAK e gera etiqueta normalmente — apontamento contém **somente** blisters efetivamente embalados.

9.2 Reimpressão de etiqueta

Pelo painel diário (generateBarcodeByBoxId): reutiliza generateBarcode() com dados do banco. Não passa pela rota /api/op-jerp/barcode (sem auditoria automática). Pode gerar novo apontamento no JERP se chamado novamente.

9.3 Conclusão da OP

opCompletionNowHandler verifica localmente: todas OpBox com packedAt e barCodeGeneratedAt preenchidos → op.status = COMPLETED. Não há chamada ao JERP para encerrar a OP.

10. Referência de arquivos

Responsabilidade	Arquivo
Cliente HTTP JERP + generateBarcode	src/shared/services/jerp/index.ts
API de etiqueta/apontamento	src/app/api/op-jerp/barcode/route.ts
Montagem array embalagens	src/usecases/op-jerp/build-jerp-embalagem-apontamento.ts
Resumo embalado (fonte autoritativa)	src/usecases/op-jerp/get-authoritative-box-packed-summary.ts
Query blisters embalados	src/usecases/op-jerp/get-packed-blisters-by-box.ts
Tela de inspeção + printTag	src/app/op/[opId]/page.tsx
Persistência caixa/blisters + saveTagId	src/app/op/[opId]/actions.tsx
Reimpressão painel diário	src/features/daily-op-box-table/actions.ts
DTO embalagem	src/types/dtos/jerp-embalagem-apontamento-dto.ts
Testes integração payload	src/usecases/op-jerp/jerp-payload-integration.test.ts
Script auditoria payload	scripts/trace-jerp-payload.ts

11. Exemplo completo — OP 76691, caixa de 48 peças

Contexto: 8 blisters × 6 peças.

1. Operador inspeciona caixa → blister 1 (QR 0710380009, 6 peças) → ... → blister 8
2. Sistema grava 8 registros OpBoxBlister com packedAt e códigos QR
3. getAuthoritativeBoxPackedSummary retorna quantity: 48, 8 blisters

4. POST /ordemproducao:

```
{
  "id": 430505,
  "quantidadeApontada": 48,
  "userName": "operador@gde.com.br",
  "embalagens": [
    { "barcode": "0710380009" },
    { "barcode": "0710380006" },
    { "barcode": "0710380005" },
    { "barcode": "0710380010" },
    { "barcode": "0710380007" },
    { "barcode": "0710380008" },
    { "barcode": "0710380004" },
    { "barcode": "0710380003" }
  ]
}
```

5. JERP responde com idBarras, quantidadePendente: 1032, pdfBase64

6. Sistema salva barCode na caixa e imprime etiqueta

7. Operador passa para a caixa 2

Documento gerado a partir da implementação em gde-insp-embalagem — apontamento JERP com códigos de barra em embalagens[]. Para simular payloads reais: `npx ts-node scripts/trace-jerp-payload.ts`